

Classes, Encapsulamento, Herança, Polimorfismo e Boas Práticas

COMP0496 – Programação A
Programação Orientada a Objetos

Prof. Dr. André Yoshiaki Kashiwabara

2 de junho de 2026

- Aquecimento — Tudo em Python é Objeto
- O Problema — A Sopa de Variáveis
- A Solução — Classes e Objetos
- Atributos de Classe vs Instância
- Composição: Objetos Dentro de Objetos
- Encapsulamento
- @property — Acesso Controlado
- Métodos Especiais (Dunders)
- Prática — Classe Fracao
- Herança Simples
- Sobrescrita de Métodos
- Polimorfismo e Duck Typing
- Herança vs. Composição
- Herança Múltipla e MRO
- Classes Abstratas
- @dataclass — Eliminando Boilerplate
- @classmethod e @staticmethod
- Princípios SOLID (Introdução)
- Padrões de Projeto
- Resumo do Módulo

Aquecimento — Tudo em Python é Objeto

Você já usa objetos sem saber! Veja o que `type()` retorna:

```
1 >>> type("Oi")
2 <class 'str'>
3 >>> type([1, 2, 3])
4 <class 'list'>
5 >>> type(42)
6 <class 'int'>
```

Cada valor em Python pertence a uma **classe** — um tipo que define quais operações são possíveis.

Métodos são funções que “pertencem” a um objeto:

```
1 >>> "programacao".upper()  
2 'PROGRAMACAO'  
3  
4 >>> frutas = ["banana", "manga"]  
5 >>> frutas.append("uva")  
6 >>> frutas  
7 ['banana', 'manga', 'uva']
```

Ponto-chave

POO = criar **seus próprios tipos** (classes) com seus próprios métodos e atributos.

O Problema — A Sopa de Variáveis

Você abriu um food truck e precisa de um sistema para gerenciar o cardápio.

Primeira tentativa: variáveis soltas.

```
1 lanche1_nome = "X-Tudo"
2 lanche1_preco = 18.50
3 lanche1_ingredientes = "pao, hamburguer, queijo, ovo, bacon"
4
5 lanche2_nome = "X-Salada"
6 lanche2_preco = 14.00
7 lanche2_ingredientes = "pao, hamburguer, queijo, alface, tomate"
```

- **Dados soltos:** nome, preço e ingredientes não estão conectados — qualquer variável pode ser alterada isoladamente
- **Não escala:** com 50 lanches, seriam 150 variáveis!
- **Erros de digitação:** `lanche1_preco` vs. `lanche1_prço` — sem aviso
- **Sem comportamento:** não há como pedir a um lanche que “se descreva”

Analogia

É como anotar pedidos em guardanapos separados: fácil perder, fácil trocar, impossível de organizar.

A Solução — Classes e Objetos

Classe = Receita

- Define *quais* informações um lanche tem
- Define *o que* um lanche pode fazer
- É um molde, um modelo
- Existe uma única vez no código

Objeto = Lanche Pronto

- Preenche a receita com dados reais
- Cada objeto é independente
- Pode haver muitos objetos da mesma classe
- Vive na memória do programa

```
1 class Lanche:
2     """Representa um lanche do cardapio."""
3
4     def __init__(self, nome, preco, ingredientes):
5         self.nome = nome
6         self.preco = preco
7         self.ingredientes = ingredientes
8
9     def descrever(self):
10        return f"{self.nome} - R${self.preco:.2f}"
```

- `__init__` é o **construtor** — é chamado automaticamente ao criar um objeto
- `self` é uma referência ao **próprio objeto** que está sendo criado
- `self.nome = nome` cria um **atributo de instância** — cada objeto terá o seu

Atenção

`self` **sempre** é o primeiro parâmetro de qualquer método, mas **não** é passado na chamada — Python cuida disso.

```
1 xtudo = Lanche("X-Tudo", 18.50, "pao, hamburguer, queijo, ovo, bacon")
2 xsalada = Lanche("X-Salada", 14.00, "pao, hamburguer, queijo, alface")
3
4 print(xtudo.descrever()) # X-Tudo - R$18.50
5 print(xsalada.nome) # X-Salada
6 print(xsalada.preco) # 14.0
```

Cada objeto é independente: alterar `xtudo.preco` não afeta `xsalada.preco`.

Sem OOP

```
1 lanche1_nome = "X-Tudo"
2 lanche1_preco = 18.50
3 # ... e mais 148 variaveis
```

Com OOP

```
1 xtudo = Lanche("X-Tudo", 18.50,
2             "pao, hamburger...")
3 # 1 variavel = 1 lanche completo
```

Dados e comportamento ficam **juntos** — é isso que “orientação a objetos” significa.

Conceito	Significado
Classe	Modelo/receita que define atributos e métodos
Objeto	Instância concreta de uma classe
<code>__init__</code>	Método construtor — inicializa o objeto
<code>self</code>	Referência ao próprio objeto
Atributo	Variável que pertence ao objeto
Método	Função que pertence à classe

```
1 class Lanche:
2     def __init__(self, nome, preco, ingredientes):
3         self.nome = nome
4         self.preco = preco
5         self.ingredientes = ingredientes
6
7     def descrever(self):
8         return f"{self.nome} - R${self.preco:.2f}"
9
10    def aplicar_desconto(self, percentual):
11        self.preco *= (1 - percentual / 100)
12
13    def tem_ingrediente(self, ingrediente):
14        return ingrediente in self.ingredientes
```



```
1 xtudo = Lanche("X-Tudo", 18.50, "pao, hamburguer, queijo, ovo")
2
3 print(xtudo.descrever()) # X-Tudo - R$18.50
4
5 xtudo.aplicar_desconto(10)
6 print(xtudo.descrever()) # X-Tudo - R$16.65
7
8 print(xtudo.tem_ingrediente("ovo")) # True
9 print(xtudo.tem_ingrediente("bacon")) # False
```

O objeto **sabe** como se descrever, como aplicar desconto e como verificar ingredientes — o comportamento está *dentro* do objeto.

Atributos de Classe vs Instância

	Atributo de Classe	Atributo de Instância
Onde define	Dentro da classe, fora de métodos	Dentro de <code>__init__</code>
Compartilhado	Sim, por todos os objetos	Não, cada objeto tem o seu
Acesso	<code>Classe.atributo</code>	<code>self.atributo</code>
Exemplo	Taxa de serviço (10%)	Nome do lanche

Exemplo: Atributo de Classe



```
1 class Lanche:
2     taxa_servico = 0.10 # compartilhado por TODOS os lanches
3
4     def __init__(self, nome, preco, ingredientes):
5         self.nome = nome
6         self.preco = preco
7         self.ingredientes = ingredientes
8
9     def preco_com_taxa(self):
10        return self.preco * (1 + Lanche.taxa_servico)
```

```
1 xtudo = Lanche("X-Tudo", 18.50, "pao, hamburguer, queijo")
2 print(xtudo.preco_com_taxa()) # 20.35
3
4 Lanche.taxa_servico = 0.15 # altera para TODOS
5 print(xtudo.preco_com_taxa()) # 21.275
```

```
1 class Lanche:
2     extras = [] # PERIGO! Lista compartilhada
3
4     def __init__(self, nome):
5         self.nome = nome
6
7     def adicionar_extra(self, extra):
8         self.extras.append(extra)
```

```
1 a = Lanche("X-Tudo")
2 b = Lanche("X-Salada")
3 a.adicionar_extra("bacon")
4 print(b.extras) # ['bacon'] -- b tambem tem bacon?!
```

```
1 class Lanche:
2     def __init__(self, nome):
3         self.nome = nome
4         self.extras = [] # CORRETO: cada objeto tem sua lista
5
6     def adicionar_extra(self, extra):
7         self.extras.append(extra)
```

```
1 a = Lanche("X-Tudo")
2 b = Lanche("X-Salada")
3 a.adicionar_extra("bacon")
4 print(a.extras) # ['bacon']
5 print(b.extras) # [] -- independente!
```

Regra de Ouro

Atributos mutáveis (listas, dicionários, conjuntos) devem ser criados em `__init__`, **nunca** como atributos de classe.

Composição: Objetos Dentro de Objetos

Um Cardápio contém uma **lista de objetos** Lanche.

Isso é chamado de **composição**: um objeto é composto por outros objetos.

Analogia

O cardápio do food truck não é um lanche — ele *contém* lanches. A relação é “tem-um” (has-a), não “é-um” (is-a).


```
1 class Cardapio:
2     """Gerencia a colecao de lanches do food truck."""
3
4     def __init__(self, nome_food_truck):
5         self.nome_food_truck = nome_food_truck
6         self.lanches = [] # lista de objetos Lanche
7
8     def adicionar(self, lanche):
9         self.lanches.append(lanche)
10
11     def mostrar(self):
12         print(f"=== Cardapio: {self.nome_food_truck} ===")
13         for lanche in self.lanches:
14             print(f" {lanche.descrever()}")
```

```
1 class Cardapio:
2     # ... (continuacao)
3
4     def preco_medio(self):
5         if not self.lanches:
6             return 0
7         total = sum(l.preco for l in self.lanches)
8         return total / len(self.lanches)
9
10    def mais_caro(self):
11        if not self.lanches:
12            return None
13        return max(self.lanches, key=lambda l: l.preco)
```

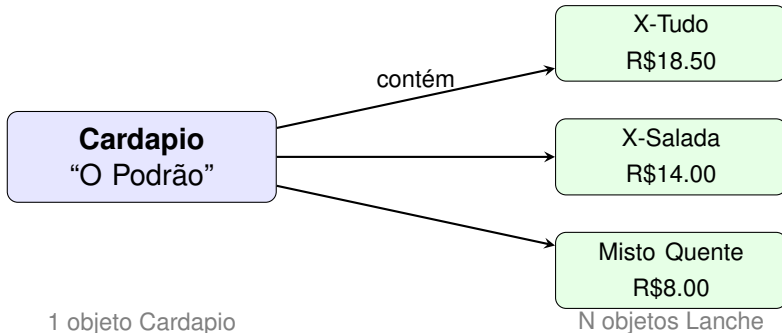
```
1 cardapio = Cardapio("O Podrao")
2 cardapio.adicionar(Lanche("X-Tudo", 18.50, "pao, hamburguer..."))
3 cardapio.adicionar(Lanche("X-Salada", 14.00, "pao, alface..."))
4 cardapio.adicionar(Lanche("Misto Quente", 8.00, "pao, presunto"))
5
6 cardapio.mostrar()
```

Saída:

```
=== Cardapio: O Podrao ===
X-Tudo - R$18.50
X-Salada - R$14.00
Misto Quente - R$8.00
```

```
1 print(f"Preço medio: R${cardapio.preco_medio():.2f}")
2 # Preço medio: R$13.50
3
4 campeão = cardapio.mais_caro()
5 print(f"Mais caro: {campeao.descrever()}")
6 # Mais caro: X-Tudo - R$18.50
```

Note como `mais_caro()` retorna um **objeto** `Lanche`, e podemos chamar `descrever()` nele — objetos conversam entre si!



Encapsulamento

```
1 xtudo = Lanche("X-Tudo", 18.50)
2 xtudo.preco = -999 # nenhum erro!
3 xtudo.preco = "gratis" # nenhum erro!
4 print(xtudo.preco) # "gratis"
```

Problema

Qualquer código externo pode colocar o objeto em um estado que não faz sentido. Um preço negativo ou textual viola as regras do nosso domínio.

Como proteger os dados internos de um objeto?

Pense na caixa registradora do Podrão:

- O **visor** mostra o total (acesso público)
- A **gaveta** só abre com a chave do operador (acesso restrito)
- Ninguém coloca a mão diretamente no dinheiro

Encapsulamento

Esconder os detalhes internos de um objeto e expor apenas uma interface controlada.

Python **não tem** modificadores de acesso como Java. Usa-se **convenções de nomenclatura**:

Nome	Convenção	Significado
nome	público	Acesso livre
<code>_nome</code>	protegido	“Use com cuidado”
<code>__nome</code>	privado	Name mangling (dificulta acesso)

Filosofia Python

“We’re all consenting adults here” — a linguagem confia no programador, mas oferece mecanismos para sinalizar intenção.

Atributos com `__` (duplo sublinhado) sofrem **name mangling**:

```
1 class CaixaRegistradora:
2     def __init__(self):
3         self.__saldo = 0.0
4
5     def receber_pagamento(self, valor):
6         if valor > 0:
7             self.__saldo += valor
8
9     def ver_saldo(self):
10        return self.__saldo
```

```
1 caixa = CaixaRegistradora()
2 caixa.receber_pagamento(25.00)
3 print(caixa.ver_saldo()) # 25.0
4
5 print(caixa.__saldo)
6 # AttributeError: 'CaixaRegistradora' object
7 # has no attribute '__saldo'
```

Name mangling por dentro

O Python renomeia `__saldo` para `_CaixaRegistradora__saldo`.

```
1 # Funciona, mas NUNCA faça isso!
2 print(caixa._CaixaRegistradora__saldo) # 25.0
```

@property — Acesso Controlado

Em Java, usamos `getPreco()` e `setPreco()`. Em Python, isso fica verboso e pouco natural:

```
1  # Estilo Java em Python (evite!)
2  lanche.get_preco()
3  lanche.set_preco(20.0)
4
5  # Estilo Pythonico (queremos isto)
6  lanche.preco # leitura
7  lanche.preco = 20.0 # escrita com validacao
```

A solução: `@property`.

```
1 class Lanche:
2     def __init__(self, nome, preco):
3         self._nome = nome
4         self.preco = preco # usa o setter!
5
6     @property
7     def preco(self):
8         return self._preco
9
10    @preco.setter
11    def preco(self, valor):
12        if valor < 0:
13            raise ValueError("Preco negativo!")
14        if valor > 500:
15            raise ValueError("Preco acima do limite!")
16        self._preco = valor
```

```
1 xtudo = Lanche("X-Tudo", 18.50)
2 print(xtudo.preco) # 18.5 (chama o getter)
3
4 xtudo.preco = 22.0 # OK (chama o setter)
5 xtudo.preco = -10 # ValueError: Preço negativo!
6 xtudo.preco = 1000 # ValueError: Preço acima do limite!
```

Vantagem

O código externo usa sintaxe de atributo, mas internamente há validação. Mudança transparente.

Use @property **sem** definir o setter:

```
1 class Pedido:
2     def __init__(self):
3         self._itens = []
4
5     def adicionar(self, lanche, qtd=1):
6         self._itens.append((lanche, qtd))
7
8     @property
9     def total(self):
10         return sum(l.preco * q for l, q in self._itens)
```

```
1 pedido = Pedido()
2 pedido.adicionar(xtudo, 2)
3 print(pedido.total) # 37.0
4 pedido.total = 0 # AttributeError!
```


Cuidado!

Não use o mesmo nome para a property e o atributo interno.

```
1 class Errado:
2     @property
3     def preco(self):
4         return self.preco # chama a si mesmo!
5
6     @preco.setter
7     def preco(self, valor):
8         self.preco = valor # chama a si mesmo!
9         # RecursionError!
```

Solução: armazene em `self._preco` (com prefixo).

Métodos Especiais (Dunders)

Métodos com duplo sublinhado: `__nome__`

São chamados automaticamente pelo Python em situações específicas:

Você escreve	Python chama
<code>print(obj)</code>	<code>obj.__str__()</code>
<code>repr(obj)</code>	<code>obj.__repr__()</code>
<code>obj1 == obj2</code>	<code>obj1.__eq__(obj2)</code>
<code>obj1 < obj2</code>	<code>obj1.__lt__(obj2)</code>
<code>len(obj)</code>	<code>obj.__len__()</code>
<code>hash(obj)</code>	<code>obj.__hash__()</code>
<code>obj1 + obj2</code>	<code>obj1.__add__(obj2)</code>

```
1 class Lanche:
2     def __init__(self, nome, preco):
3         self._nome = nome
4         self._preco = preco
5
6 xtudo = Lanche("X-Tudo", 18.5)
7 print(xtudo)
8 # <__main__.Lanche object at 0x7f3b2c> (feio!)
```

Agora com dunder:

```
1     def __str__(self):
2         return f"{self._nome} - R${self._preco:.2f}"
3
4     def __repr__(self):
5         return f"Lanche('{self._nome}', {self._preco})"
```

```
1 print(xtudo) # X-Tudo - R$18.50 (amigavel)
2 repr(xtudo) # Lanche('X-Tudo', 18.5) (tecnico)
```

Por padrão, dois objetos com os mesmos dados são **diferentes**:

```
1 a = Lanche("X-Tudo", 18.5)
2 b = Lanche("X-Tudo", 18.5)
3 print(a == b) # False (compara identidade!)
```

Solução:

```
1 def __eq__(self, other):
2     if not isinstance(other, Lanche):
3         return NotImplemented
4     return (self._nome == other._nome
5           and self._preco == other._preco)
```

```
1 print(a == b) # True
```

```
1 def __lt__(self, other):
2     if not isinstance(other, Lanche):
3         return NotImplemented
4     return self._preco < other._preco
```

```
1 cardapio = [
2     Lanche("X-Tudo", 18.5),
3     Lanche("Misto", 8.0),
4     Lanche("Burgao", 22.0),
5 ]
6 for l in sorted(cardapio):
7     print(l)
8     # Misto - R$8.00
9     # X-Tudo - R$18.50
10    # Burgao - R$22.00
```

Ao definir `__eq__`, o Python desabilita `__hash__`:

```
1 lanches = {Lanche("X-Tudo", 18.5)}  
2 # TypeError: unhashable type: 'Lanche'
```

Solução:

```
1 def __hash__(self):  
2     return hash((self._nome, self._preco))
```

Regra de ouro

Se `a == b`, então `hash(a) == hash(b)`. Use os mesmos atributos em ambos os métodos.

```
1 class Pedido:
2     def __init__(self):
3         self._itens = []
4
5     def adicionar(self, lanche, qtd=1):
6         self._itens.append((lanche, qtd))
7
8     def __len__(self):
9         return sum(q for _, q in self._itens)
```

```
1 pedido = Pedido()
2 pedido.adicionar(Lanche("X-Tudo", 18.5), 2)
3 pedido.adicionar(Lanche("Misto", 8.0), 1)
4 print(len(pedido)) # 3
```


Método	Operação	Quando usar
<code>__init__</code>	Construtor	Sempre
<code>__str__</code>	<code>print()</code> , <code>str()</code>	Saída amigável
<code>__repr__</code>	<code>repr()</code> , terminal	Debug, logs
<code>__eq__</code>	<code>==</code>	Comparação por valor
<code>__lt__</code>	<code><</code> , <code>sorted()</code>	Ordenação
<code>__hash__</code>	<code>set</code> , <code>dict</code>	Após definir <code>__eq__</code>
<code>__len__</code>	<code>len()</code>	Coleções
<code>__add__</code>	<code>+</code>	Soma/concatenação
<code>__sub__</code>	<code>-</code>	Subtração
<code>__mul__</code>	<code>*</code>	Multiplicação

Prática — Classe Fracao

Vamos construir uma classe completa que usa **tudo** que aprendemos:

- Validação no construtor (denominador $\neq 0$)
- Simplificação automática via MDC
- Properties somente-leitura
- `__str__`, `__repr__`
- `__eq__`, `__lt__`, `__hash__`
- `__add__`, `__sub__`, `__mul__`

```
1 from math import gcd
2
3 class Fracao:
4     def __init__(self, num, den=1):
5         if den == 0:
6             raise ValueError("Denominador nao pode ser zero!")
7         # sinal sempre no numerador
8         if den < 0:
9             num, den = -num, -den
10        divisor = gcd(abs(num), den)
11        self._num = num // divisor
12        self._den = den // divisor
13
14    @property
15    def num(self):
16        return self._num
17
18    @property
19    def den(self):
20        return self._den
```

```
1     def __str__(self):
2         if self._den == 1:
3             return str(self._num)
4         return f"{self._num}/{self._den}"
5
6     def __repr__(self):
7         return f"Fracao({self._num}, {self._den})"
```

```
1 print(Fracao(4, 6)) # 2/3
2 print(Fracao(6, 3)) # 2
3 repr(Fracao(4, 6)) # Fracao(2, 3)
```

```
1  def __eq__(self, other):
2      if not isinstance(other, Fracao):
3          return NotImplemented
4      return (self._num == other._num
5              and self._den == other._den)
6
7  def __lt__(self, other):
8      if not isinstance(other, Fracao):
9          return NotImplemented
10     return self._num * other._den < other._num * self._den
11
12  def __hash__(self):
13     return hash((self._num, self._den))
```

```
1  def __add__(self, other):
2      if not isinstance(other, Fracao):
3          return NotImplemented
4      return Fracao(self._num * other._den
5                     + other._num * self._den,
6                     self._den * other._den)
7
8  def __sub__(self, other):
9      if not isinstance(other, Fracao):
10         return NotImplemented
11     return Fracao(self._num * other._den
12                  - other._num * self._den,
13                  self._den * other._den)
14
15  def __mul__(self, other):
16      if not isinstance(other, Fracao):
17         return NotImplemented
18     return Fracao(self._num * other._num,
19                  self._den * other._den)
```

```
1 a = Fracao(1, 2)
2 b = Fracao(1, 3)
3
4 print(a + b) # 5/6
5 print(a - b) # 1/6
6 print(a * b) # 1/6
7 print(a == Fracao(2, 4)) # True (simplificado!)
```

```
1 fracs = [Fracao(3,4), Fracao(1,2), Fracao(2,3)]
2 print(sorted(fracs)) # [1/2, 2/3, 3/4]
```

```
1 # set() funciona porque temos __eq__ e __hash__
2 s = {Fracao(1,2), Fracao(2,4), Fracao(3,6)}
3 print(s) # {1/2}
```


Herança Simples

No food truck **O Podrão**, temos três lanches com código repetido:

```
1 class XTudo:
2     def __init__(self, preco):
3         self.preco = preco
4         self.nome = "X-Tudo"
5
6     def descrever(self):
7         return f"{self.nome}:
8             R${self.preco:.2f}"
9
10    def preparar(self):
11        print("Fritando hambú
12            rguer...")
```

```
1 class Dogao:
2     def __init__(self, preco):
3         self.preco = preco
4         self.nome = "Dogão"
5
6     def descrever(self):
7         return f"{self.nome}:
8             R${self.preco:.2f}"
9
10    def preparar(self):
11        print("Montando hot-
12            dog...")
```

```
1 class Acai:
2     def __init__(self, preco):
3         self.preco = preco
4         self.nome = "Açaí"
5
6     def descrever(self):
7         return f"{self.nome}:
8             R${self.preco:.2f}"
9
10    def preparar(self):
11        print("Batendo açaí...")
```

Violação do Princípio DRY

Don't Repeat Yourself — código duplicado é fonte de bugs.

- `__init__` e `descrever` são **idênticos** nas três classes
- Se precisar adicionar `calorias`: alterar **três** classes
- Se corrigir um bug em `descrever`: corrigir em **três** lugares
- E se o cardápio crescer para 15 itens?

Solução

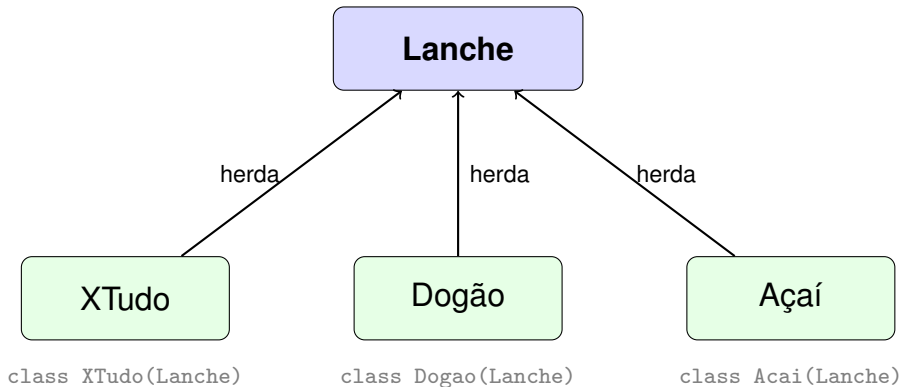
Extrair o que é **comum** para uma classe base $\Rightarrow$ **Herança**.

Herança modela o relacionamento “é um” (*is-a*):

Afirmação	Verdade?	Herança?
XTudo é um Lanche	✓	Sim
Dogão é um Lanche	✓	Sim
Açaí é um Lanche	✓	Sim
Pedido é uma CaixaRegistradora	×	Não
Pedido <i>tem</i> Lanches	✓	Composição

Regra de Ouro

Se “B é um A” faz sentido no domínio $\Rightarrow \text{class } B(A)$.



```
1 class Lanche:
2     def __init__(self, nome, preco):
3         self.nome = nome
4         self.preco = preco
5
6     def descrever(self):
7         return f"{self.nome}: R${self.preco:.2f}"
8
9     def preparar(self):
10        print(f"Preparando {self.nome}...")
```

O que mudou?

Todo código comum agora vive em **um único lugar**.

```
1 class XTudo(Lanche):
2     def __init__(self, preco, molho="especial"):
3         super().__init__("X-Tudo", preco)
4         self.molho = molho
5
6     def preparar(self):
7         print(f"Fritando hambúrguer com molho {self.molho}...")
```

```
1 class Dogao(Lanche):
2     def __init__(self, preco, tamanho="grande"):
3         super().__init__("Dogão", preco)
4         self.tamanho = tamanho
5
6     def preparar(self):
7         print(f"Montando hot-dog {self.tamanho}...")
```

```
1 x = XTudo(25.0)
2 d = Dogao(18.0)
3
4 # descrever() é herdado de Lanche --- funciona sem reescrever!
5 print(x.descrever()) # X-Tudo: R$25.00
6 print(d.descrever()) # Dogão: R$18.00
7
8 # preparar() foi sobrescrito em cada subclasse
9 x.preparar() # Fritando hambúrguer com molho especial...
10 d.preparar() # Montando hot-dog grande...
```

Regra

Se a subclasse **não** redefine um método, usa o da superclasse.


```
1 x = XTudo(25.0)
2
3 isinstance(x, XTudo) # True
4 isinstance(x, Lanche) # True (XTudo EH UM Lanche)
5 isinstance(x, Dogao) # False
6
7 type(x) == XTudo # True
8 type(x) == Lanche # False (type() é exato)
```

Verificação	<code>isinstance</code>	<code>type() ==</code>
Considera herança?	Sim	Não
Uso recomendado	Geral	Raro

Sobrescrita de Métodos

Sobrescrita (*override*): a subclasse fornece sua própria versão de um método herdado.

```
1 class Acai(Lanche):
2     def __init__(self, preco, complementos=None):
3         super().__init__("Açaí", preco)
4         self.complementos = complementos or ["granola", "banana"]
5
6     def preparar(self):
7         # Sobrescreve completamente o método de Lanche
8         print(f"Batendo açaí com {', '.join(self.complementos)}...")
```

Atenção

O método da subclasse **substitui** o da superclasse — não são acumulados.

Quando queremos **estender** (não substituir) o comportamento do pai:

```
1 class XTudoEspecial(XTudo):
2     def __init__(self, preco, extras=None):
3         super().__init__(preco, molho="da casa")
4         self.extras = extras or ["bacon", "ovo"]
5
6     def descrever(self):
7         base = super().descrever() # chama Lanche.descrever()
8         return f"{base} + {' , '.join(self.extras)}"
```

```
1 xe = XTudoEspecial(32.0)
2 print(xe.descrever())
3 # X-Tudo: R$32.00 + bacon, ovo
```

```
1 class Coxinha(Lanche):
2     def __init__(self, preco):
3         # ESQUECEU super().__init__() !!!
4         self.recheio = "frango"
5
6 c = Coxinha(8.0)
7 c.recheio # "frango" --- OK
8 c.descrever() # AttributeError: 'Coxinha' has no attribute 'nome'
```

Por quê?

Sem `super().__init__()`, os atributos `nome` e `preco` nunca são criados.

Correção

Sempre chame `super().__init__()` no início do `__init__` da subclasse.

Polimorfismo e Duck Typing

Polimorfismo: objetos de tipos diferentes respondem à mesma mensagem, cada um à sua maneira.

```
1 pedido = [XTudo(25.0), Dogao(18.0), Acai(15.0)]
2
3 for item in pedido:
4     item.preparar() # cada classe executa seu próprio preparar()
```

Saída:

```
Fritando hambúrguer com molho especial...
Montando hot-dog grande...
Batendo açaí com granola, banana...
```

Essência

O código **não precisa saber** o tipo exato — basta que o objeto tenha o método.

"If it walks like a duck and quacks like a duck, then it must be a duck."

```
1 class Bebida: # NAO herda de Lanche!
2     def __init__(self, nome, preco):
3         self.nome = nome
4         self.preco = preco
5
6     def preparar(self):
7         print(f"Servindo {self.nome}...")
8
9     def descrever(self):
10        return f"{self.nome}: R${self.preco:.2f}"
```



```
1 pedido = [XTudo(25.0), Dogao(18.0), Bebida("Guaraná", 6.0)]
2
3 for item in pedido:
4     print(item.descrever())
5     item.preparar()
```

Saída:

```
X-Tudo: R$25.00
Fritando hambúrguer com molho especial...
Dogão: R$18.00
Montando hot-dog grande...
Guaraná: R$6.00
Servindo Guaraná...
```

Observação

Bebida não herda de Lanche, mas funciona no mesmo loop porque tem os mesmos métodos. Isso é **duck typing**.

Herança vs. Composição

Relacionamento	Mecanismo	Exemplo
“É um” (<i>is-a</i>)	Herança	XTudo é um Lanche
“Tem um” (<i>has-a</i>)	Composição	Pedido tem Lanches
“Tem um” (<i>has-a</i>)	Composição	Pedido tem FormaPagamento
“É um” (<i>is-a</i>)	Herança	Pix é uma FormaPagamento

```
1 class FormaPagamento:
2     def processar(self, valor):
3         raise NotImplementedError
4
5 class Pix(FormaPagamento):
6     def processar(self, valor):
7         print(f"Pix de R${valor:.2f} confirmado!")
8
9 class Pedido:
10     def __init__(self, pagamento):
11         self.itens = [] # composição: tem lanches
12         self.pagamento = pagamento # composição: tem forma de pgto
13
14     def adicionar(self, item):
15         self.itens.append(item)
16
17     def fechar(self):
18         total = sum(i.preco for i in self.itens)
19         self.pagamento.processar(total)
```

```
1 pix = Pix()
2 pedido = Pedido(pix)
3 pedido.adicionar(XTudo(25.0))
4 pedido.adicionar(Bebida("Guaraná", 6.0))
5 pedido.fechar()
6 # Pix de R$31.00 confirmado!
```

Gang of Four

“Favor composition over inheritance.”

Se você só quer **reusar** comportamento, sem relação conceitual “é um”, use composição.

Critério	Herança	Composição
Relação “é um” genuína	✓	
Apenas reusar código		✓
Trocar comportamento em runtime		✓
Hierarquia natural e estável	✓	
Flexibilidade máxima		✓

Regra Prática

Comece com composição. Use herança apenas quando o “é um” for claro e estável.

Herança Múltipla e MRO

Python permite herdar de **mais de uma classe**:

```
1 class Promocional:
2     def aplicar_desconto(self, pct):
3         self.preco *= (1 - pct / 100)
4
5 class XTudoPromo(XTudo, Promocional):
6     pass
7
8 xp = XTudoPromo(25.0)
9 xp.aplicar_desconto(10)
10 print(xp.preco) # 22.5
```


Quando há conflito, Python usa o **C3 Linearization** para decidir qual método chamar:

```
1 print(XTudoPromo.__mro__)  
2 # (XTudoPromo, XTudo, Lanche, Promocional, object)
```

Python busca o método na ordem da MRO: da esquerda para a direita.

Conselho

Herança múltipla pode causar confusão. Na prática:

- Use **mixins** simples (como Promocional)
- Prefira **composição** para combinações complexas
- Evite hierarquias diamante

Classes Abstratas

O Problema: Métodos “Esquecidos”

```
1 class Funcionario:
2     def __init__(self, nome, salario_base):
3         self.nome = nome
4         self.salario_base = salario_base
5
6     def calcular_salario(self):
7         raise NotImplementedError("Subclasse deve implementar")
```

```
1 class Atendente(Funcionario):
2     pass # esqueceu de implementar calcular_salario!
3
4 a = Atendente("Maria", 1500)
5 a.calcular_salario() # NotImplementedError --- mas só descobre AQUI!
```

Problema

O erro só aparece quando `calcular_salario()` é chamado — pode ser tarde demais.

```
1 from abc import ABC, abstractmethod
2
3 class Funcionario(ABC):
4     def __init__(self, nome, salario_base):
5         self.nome = nome
6         self.salario_base = salario_base
7
8     @abstractmethod
9     def calcular_salario(self):
10         """Cada tipo de funcionário calcula diferente."""
11         pass
```

```
1 # Tentar instanciar a classe abstrata:
2 f = Funcionario("João", 2000)
3 # TypeError: Can't instantiate abstract class Funcionario
4 # with abstract method calcular_salario
```

Vantagem

O erro ocorre na **instanciação**, não na chamada do método — falha cedo!

```
1 class Atendente(Funcionario):
2     def calcular_salario(self):
3         return self.salario_base
4
5 class Cozinheiro(Funcionario):
6     def __init__(self, nome, salario_base, adicional_insalubridade):
7         super().__init__(nome, salario_base)
8         self.adicional = adicional_insalubridade
9
10    def calcular_salario(self):
11        return self.salario_base + self.adicional
12
13 class Entregador(Funcionario):
14     def __init__(self, nome, salario_base, entregas):
15         super().__init__(nome, salario_base)
16         self.entregas = entregas
17
18    def calcular_salario(self):
19        return self.salario_base + self.entregas * 5
```

```
1 equipe = [  
2     Atendente("Maria", 1500),  
3     Cozinheiro("João", 1800, 300),  
4     Entregador("Carlos", 1200, 80),  
5     Atendente("Ana", 1500),  
6 ]  
7  
8 total_folha = 0  
9 for func in equipe:  
10     salario = func.calcular_salario()  
11     total_folha += salario  
12     print(f"{func.nome:.<20} R${salario:>8.2f}")  
13  
14 print(f"'TOTAL':.<20} R${total_folha:>8.2f}")
```

Saída:

```
Maria..... R$ 1500.00  
João..... R$ 2100.00  
Carlos..... R$ 1600.00  
Ana..... R$ 1500.00  
TOTAL..... R$ 6700.00
```

@dataclass — Eliminando Boilerplate

Uma classe simples com `__init__`, `__repr__` e `__eq__`:

```
1 class Lanche:
2     def __init__(self, nome, preco, ingredientes):
3         self.nome = nome
4         self.preco = preco
5         self.ingredientes = ingredientes
6
7     def __repr__(self):
8         return (f"Lanche(nome={self.nome!r}, "
9                 f"preco={self.preco!r}, "
10                f"ingredientes={self.ingredientes!r})")
11
12     def __eq__(self, other):
13         if not isinstance(other, Lanche):
14             return NotImplemented
15         return (self.nome == other.nome
16                 and self.preco == other.preco
17                 and self.ingredientes == other.ingredientes)
```

Problema

~20 linhas para guardar 3 campos. E se a classe tiver 8 campos?


```
1 from dataclasses import dataclass
2
3 @dataclass
4 class Lanche:
5     nome: str
6     preco: float
7     ingredientes: list[str]
```

O decorador gera automaticamente:

- `__init__` — com todos os campos como parâmetros
- `__repr__` — representação legível
- `__eq__` — comparação campo a campo

Resultado

3 linhas substituem ~20. O código expressa **o quê**, não **o como**.

```
1 @dataclass(frozen=True)
2 class ItemCardapio:
3     nome: str
4     preco: float
5     categoria: str
```

Parâmetro	Efeito	Quando usar
frozen=True	Imutável (sem setattr)	Itens de cardápio, configs
order=True	Gera <, <=, >, >=	Ordenar por campos
eq=True	Gera __eq__ (padrão)	Quase sempre
slots=True	Usa __slots__ (3.10+)	Performance

```
1 from dataclasses import dataclass, field
2
3 @dataclass
4 class Pedido:
5     cliente: str
6     itens: list[str] = field(default_factory=list)
7     observacoes: dict[str, str] = field(default_factory=dict)
8     numero: int = field(default=0, repr=False)
```

Nunca faça isso

`itens: list[str] = []` — o mesmo objeto é compartilhado entre instâncias!

`field(default_factory=list)` cria uma **nova lista** para cada instância.

```
1 @dataclass
2 class Pedido:
3     cliente: str
4     itens: list[str] = field(default_factory=list)
5
6     def adicionar(self, item: str):
7         self.itens.append(item)
8
9     def total(self, cardapio: dict[str, float]) -> float:
10         return sum(cardapio[i] for i in self.itens)
11
12     def resumo(self) -> str:
13         return f"{self.cliente}: {len(self.itens)} itens"
```

Ponto-chave

@dataclass gera o boilerplate; você adiciona a lógica de negócio.

Use quando:

- Classe é principalmente “dados com métodos”
- Precisa de `__eq__` e `__repr__`
- Muitos campos (reduz boilerplate)
- Quer imutabilidade (`frozen`)

NÃO use quando:

- `__init__` tem lógica complexa (validação pesada)
- Herança profunda (>2 níveis)
- Classe é majoritariamente comportamento
- Compatibilidade com Python <3.7

@classmethod e @staticmethod

Tipo	Decorador	1º Parâmetro	Acessa
Instância	(nenhum)	<code>self</code>	instância + classe
Classe	<code>@classmethod</code>	<code>cls</code>	apenas classe
Estático	<code>@staticmethod</code>	(nenhum)	nada da classe

Regra prática

Precisa de `self`? → método de instância. **Precisa de `cls`?** → `classmethod`. **Não precisa de nenhum?** → `staticmethod`.

```
1 @dataclass
2 class Lanche:
3     nome: str
4     preco: float
5     categoria: str
6
7     @classmethod
8     def from_csv_line(cls, linha: str) -> "Lanche":
9         nome, preco, cat = linha.strip().split(",")
10        return cls(nome, float(preco), cat)
11
12    @classmethod
13    def from_dict(cls, d: dict) -> "Lanche":
14        return cls(**d)
```

```
1 x = Lanche.from_csv_line("X-Tudo,25.90,hamburguer")
2 y = Lanche.from_dict({"nome": "Suco", "preco": 8.0,
3                       "categoria": "bebida"})
```

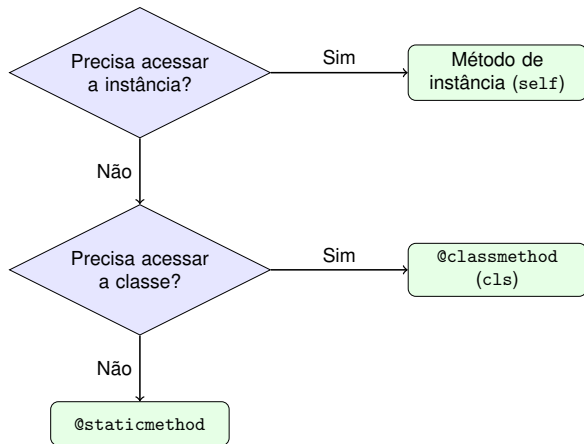


```
1 class Lanche:
2     # ...
3
4     @staticmethod
5     def calcular_taxa_servico(valor: float,
6                               percentual: float = 0.10) -> float:
7         """Taxa de serviço sobre o valor."""
8         return round(valor * percentual, 2)
```

```
1 taxa = Lanche.calcular_taxa_servico(50.00) # 5.0
2 taxa = Lanche.calcular_taxa_servico(50.00, 0.15) # 7.5
```

Quando usar

Lógica relacionada à classe, mas que não precisa de `self` nem de `cls`. Alternativa: função no módulo.



Princípios SOLID (Introdução)

Letra	Princípio	Ideia Central
S	Single Responsibility	Uma classe, uma razão para mudar
O	Open/Closed	Aberta p/ extensão, fechada p/ modificação
L	Liskov Substitution	Subtipos substituem o pai sem surpresas
I	Interface Segregation	Interfaces pequenas e coesas
D	Dependency Inversion	Dependa de abstrações, não de concretos

Veremos **S**, **O** e **L** com exemplos do Podrão.

```
1 class PedidoRuim:
2     def __init__(self, cliente, itens):
3         self.cliente = cliente
4         self.itens = itens
5
6     def calcular_total(self): ... # lógica de negócio
7     def gerar_relatorio_html(self): ... # apresentação
8     def salvar_em_arquivo(self): ... # persistência
9     def enviar_email(self): ... # comunicação
```

4 razões para mudar

Mudança no cálculo, no formato HTML, no storage, ou no e-mail — todas alteram a mesma classe.

```
1 class Pedido:
2     def calcular_total(self): ...
3
4 class RelatorioHTML:
5     def gerar(self, pedido: Pedido): ...
6
7 class Armazenamento:
8     def salvar(self, pedido: Pedido): ...
9
10 class Enviador:
11     def enviar_email(self, pedido: Pedido, dest: str): ...
```

Insight

Cada classe tem **uma única razão para mudar**. Testar cada uma é simples e independente.

```
1 class CalculadoraDesconto:
2     def calcular(self, pedido, tipo):
3         if tipo == "percentual":
4             return pedido.total() * 0.10
5         elif tipo == "fixo":
6             return 5.00
7         elif tipo == "fidelidade":
8             return pedido.total() * 0.15
9         # ... cada novo tipo exige elif
```

Violação OCP

Para adicionar “combo”, precisa **modificar** CalculadoraDesconto. A classe não está fechada para modificação.

```
1 from abc import ABC, abstractmethod
2
3 class Desconto(ABC):
4     @abstractmethod
5     def calcular(self, total: float) -> float: ...
6
7 class DescontoPercentual(Desconto):
8     def __init__(self, taxa: float):
9         self.taxa = taxa
10    def calcular(self, total: float) -> float:
11        return total * self.taxa
12
13 class DescontoFixo(Desconto):
14    def __init__(self, valor: float):
15        self.valor = valor
16    def calcular(self, total: float) -> float:
17        return min(self.valor, total)
```

Insight

Novo desconto = nova classe. Pedido não muda. **Extensão sem modificação.**


```
1 class Retangulo:
2     def __init__(self, largura, altura):
3         self._largura = largura
4         self._altura = altura
5
6     def set_largura(self, v):
7         self._largura = v
8
9     def area(self):
10        return self._largura * self._altura
11
12 class Quadrado(Retangulo):
13     def set_largura(self, v):
14         self._largura = v
15         self._altura = v # surpresa!
```

Violação LSP

Código que espera `Retangulo` e chama `set_largura(5)` seguido de `area()` obtém resultado inesperado com `Quadrado`.

Padrões de Projeto

```
1 class SemDesconto(Desconto):
2     def calcular(self, total: float) -> float:
3         return 0.0
4
5 @dataclass
6 class Pedido:
7     cliente: str
8     itens: list[str] = field(default_factory=list)
9     desconto: Desconto = field(
10         default_factory=SemDesconto)
11
12     def total(self, cardapio: dict) -> float:
13         bruto = sum(cardapio[i] for i in self.itens)
14         return bruto - self.desconto.calcular(bruto)
```

Composição

Pedido recebe a estratégia; não sabe qual é. Novo desconto = nova classe, zero mudança em Pedido.

```
1 cardapio = {"X-Tudo": 25.90, "Suco": 8.00}
2
3 p1 = Pedido("Ana", desconto=DescontoPercentual(0.10))
4 p1.adicionar("X-Tudo")
5 p1.adicionar("Suco")
6 print(p1.total(cardapio)) # 30.51
7
8 p2 = Pedido("Bruno", desconto=DescontoFixo(5.00))
9 p2.adicionar("X-Tudo")
10 print(p2.total(cardapio)) # 20.90
```

A estratégia pode ser trocada em **tempo de execução**:

```
1 p1.desconto = DescontoFixo(10.00)
```

```
1 class LancheFactory:
2     _registry: dict[str, type] = {}
3
4     @classmethod
5     def registrar(cls, nome: str, classe: type):
6         cls._registry[nome] = classe
7
8     @classmethod
9     def criar(cls, nome: str, **kwargs):
10         if nome not in cls._registry:
11             raise ValueError(f"Tipo desconhecido: {nome}")
12         return cls._registry[nome](**kwargs)
13
14 # Registro
15 LancheFactory.registrar("xtudo", XTudo)
16 LancheFactory.registrar("xsalada", XSalada)
17 LancheFactory.registrar("hamburguer", Hamburguer)
```

```
1 # Criação via string (ex: lido de arquivo/API)
2 lanche = LancheFactory.criar("xtudo", preco=25.90)
3
4 # Extensão em runtime --- sem tocar na Factory
5 class XBacon(Lanche):
6     def __init__(self, preco=28.00):
7         super().__init__("X-Bacon", preco, ["bacon"])
8
9 LancheFactory.registrar("xbacon", XBacon)
10 lanche2 = LancheFactory.criar("xbacon")
```

Vantagens

- Desacopla criação de uso
- Extensível em runtime (registrar)
- String → objeto (útil para configs, APIs, CLIs)

Resumo do Módulo

Conceito	Seção	No Podrão
Classe e objeto	Classes e Objetos	Lanche, Pedido
Atributos e métodos	Classes e Objetos	nome, preco(), adicionar()
Encapsulamento	Encapsulamento	_preco, @property
Dunders	Métodos Especiais	__str__, __eq__, __add__
Herança	Herança Simples	XTudo(Lanche), Dogao(Lanche)
Classe abstrata	Classes Abstratas	Funcionario(ABC), Desconto(ABC)
Polimorfismo	Polimorfismo	preparar() em cada subtipo
Composição	Herança vs Composição	Pedido tem lista de Lanche
@dataclass	@dataclass	Pedido, ItemCardapio
@classmethod	@classmethod	Lanche.from_csv_line()
Strategy	Padrões de Projeto	Desconto → DescontoPercentual
Factory	Padrões de Projeto	LancheFactory.criar()
SOLID (S, O, L)	Princípios SOLID	SRP, OCP, LSP

1. **Objetos agrupam dados e comportamento**

Não use listas paralelas. Crie uma classe que una o dado ao que se faz com ele.

2. **Prefira composição a herança**

Herança cria acoplamento forte. Composição permite trocar comportamento em runtime (Strategy).

3. **Cada classe, uma responsabilidade + extensão sem modificação**

SRP + OCP. Classes pequenas, coesas, extensíveis via polimorfismo.

Se lembrar apenas disso...

Estes três princípios guiam a maior parte das decisões de design em POO.